

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	K. Pujitha	Reg. No.:	192524149
Section / Batch:	2nd Year (AI & DS)	Date:	02/09/2026

Code of ConductI K. Pujitha (192524149) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Pujitha**Instructions**

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

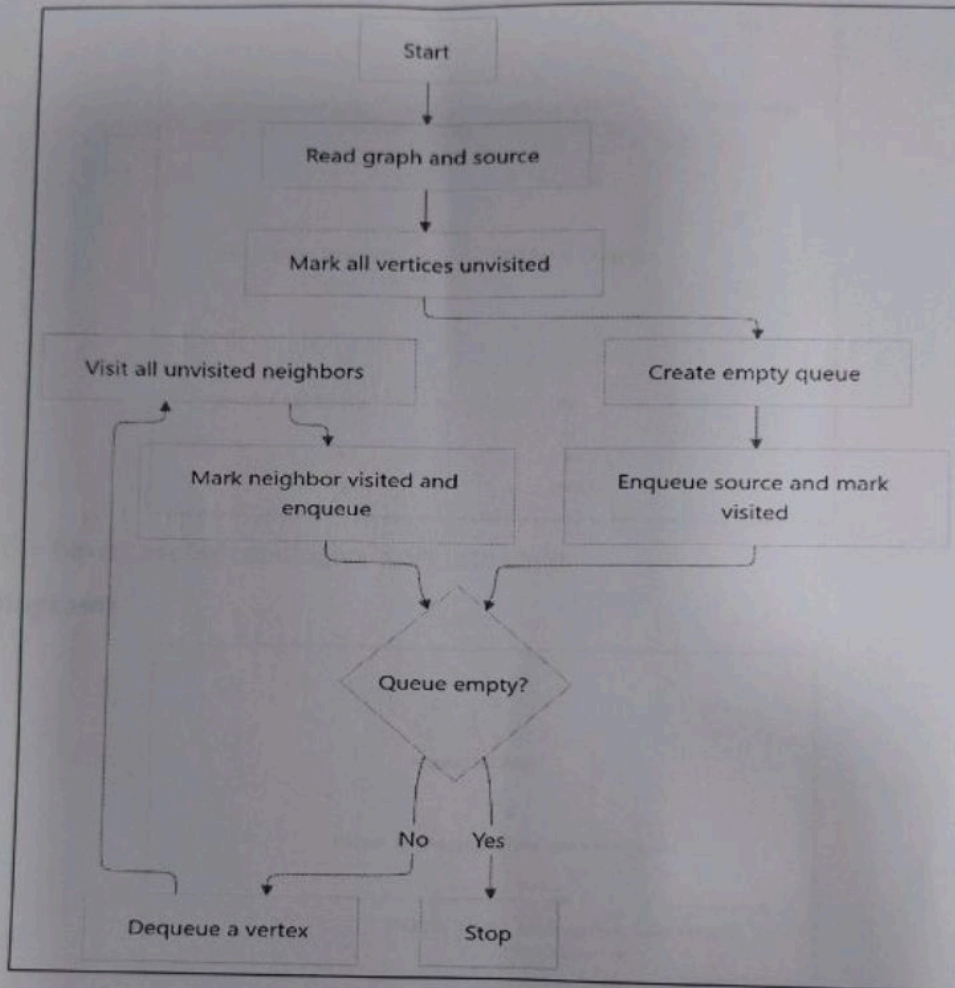
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A – Flowchart-To-Code Conversion

1. Convert the flowchart for Breadth First Search traversal into code.

(20 Marks)

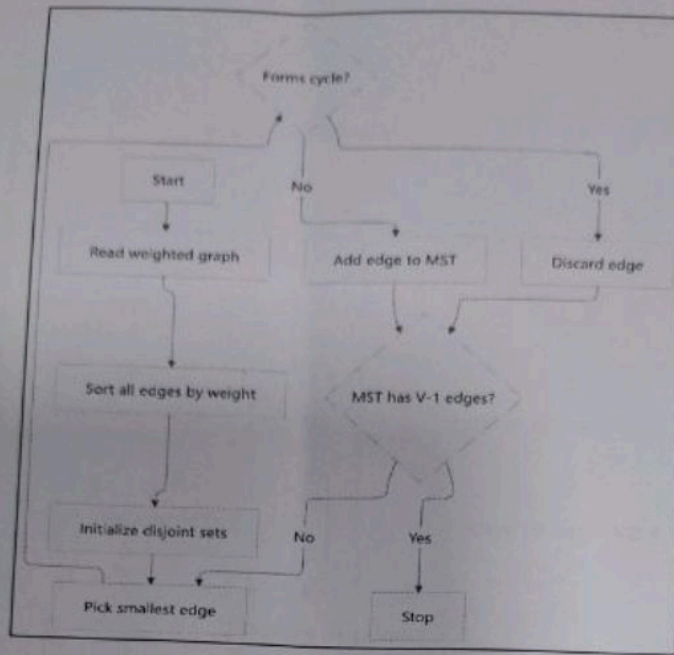
Flowchart Diagram:



2. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

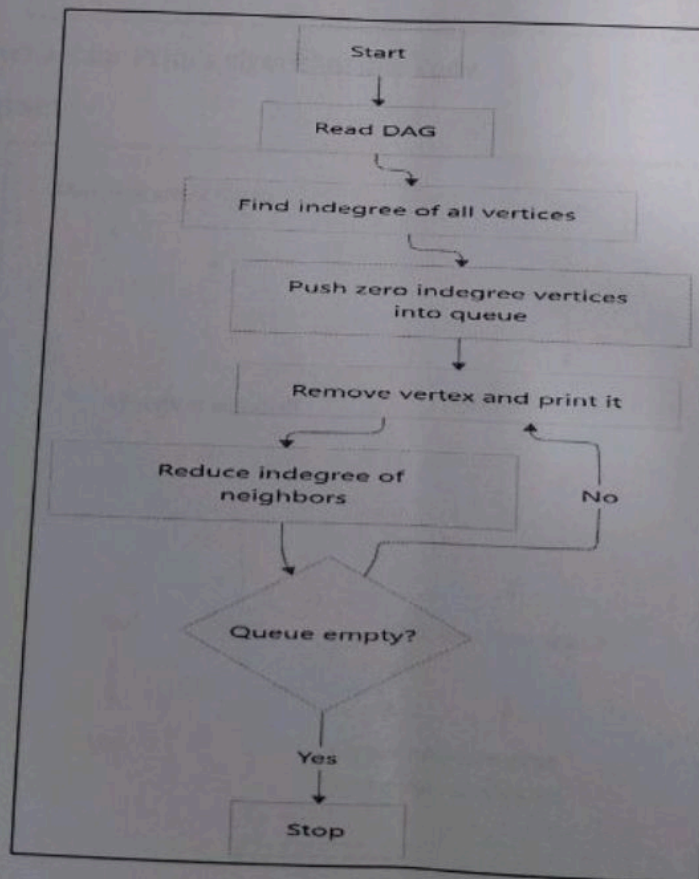
Flowchart Diagram:



3. Convert the flowchart for topological sort into code.

(20 Marks)

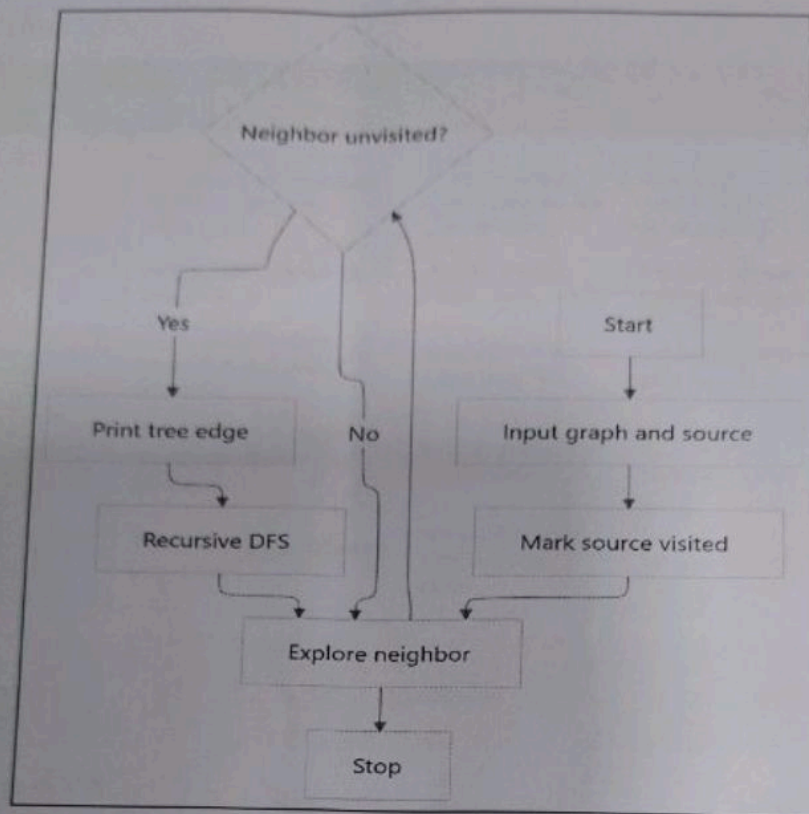
Flowchart Diagram:



4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

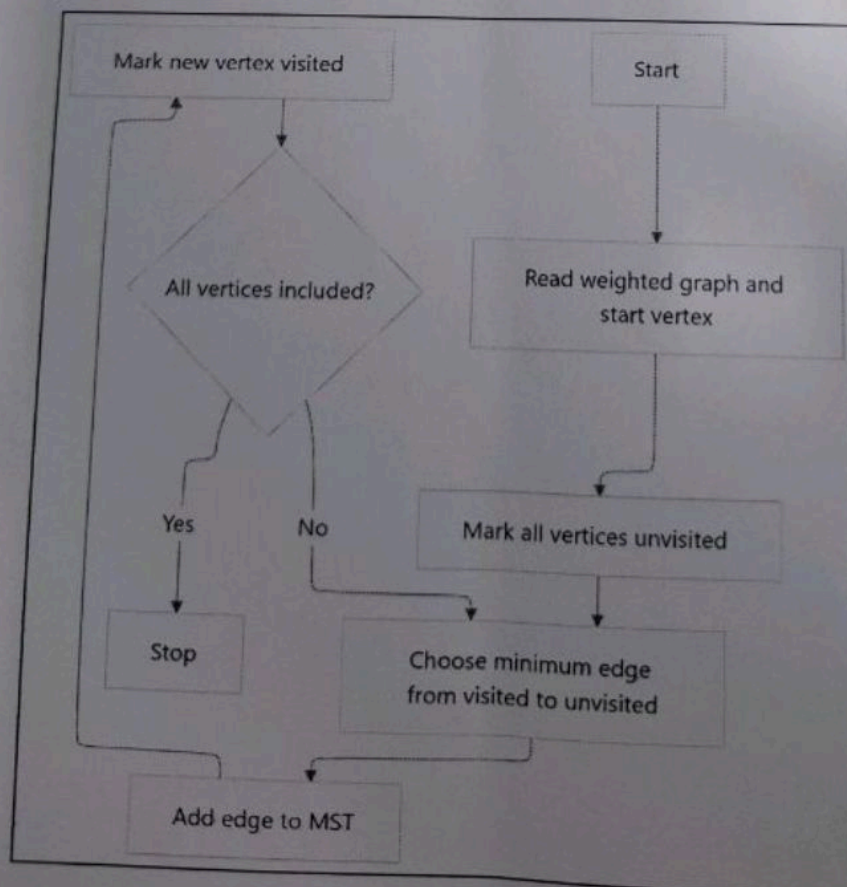
Flowchart Diagram:



5. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge

Course Coordinator

Program Director

Signature: _____

Signature: _____

Signature: _____

Date: _____

Date: _____

Date: _____

① Convert the flowchart for Breadth First search traversal into code.

Program

```
void BFS (int, s)
{
    int q[20], f=0, r=0, v, i;
    q[r++] = s;
    visited[s] = 1;
    while (f < r)
    {
        v = q[f++];
        printf ("%d", v);
        for (i=0; i < n; i++)
            if (graph[v][i] && !visited[i])
            {
                visited[i] = 1;
                q[r++] = i;
            }
    }
}
```

② Convert the flowchart for Kruskal's algorithm into code.

Program :-

```
void kruskal ()
{
```



```

int q, u, v;
SortEdges();
for (i=0; i<E; i++)
{
    u = find (edge[i].u);
    v = find (edge[i].v);
    if (u!=v)
    {
        Print f ("%d - %d \n",
            edge[i].u, edge[i].v);
        Parent[u]=v;
    }
}
}

```

3) Convert the flowchart for topological sort into code.

Program:

```

Void Topological ()
{
    int q[20], f=0, r=0, i, v;
    for (i=0; i<n; i++)
        if (indegree[i]==0)
            q[r++]=i;
    while (f<r)
    {
        v = q[f++];
    }
}

```

```

    printf ("%d", v);
    for (i = 0; i < n; i++)
    {
        indegree [i]--;
        if (indegree [i] == 0)
            q [s++] = i;
    }
}
}

```

4) Convert the flowchart for generating a DFS tree into code.

Program:

```

Void DFS (int v)
{
    int i;
    visited [v] = 1;
    for (i = 0; i < n; i++)
        if (graph [v][i] && !visited [i])
        {
            printf ("%d - %d \n", v, i);
            DFS (i);
        }
}
}

```


5) Convert - the flowchart for Prim's algorithm into code.

Program:-

```
void Prim()
{
    int i, j, u, v, min;
    Selected[0] = 1;
    for (int e = 0; e < n-1; e++)
    {
        min = 999;
        for (i = 0; i < n; i++)
            if (Selected[i])
                for (j = 0; j < n; j++)
                    if (!Selected[j] &&
                        graph[i][j] &&
                        graph[i][j] < min)
                    {
                        min = graph[i][j];
                        u = i; v = j;
                    }
        printf("%d - %d : %d\n", u, v, min);
        Selected[v] = 1;
    }
}
```

3
3
Ujjay